

HIDRARA

CONEXÕES E EQUIPAMENTOS HIDRÁULICOS

Manual de Instalação & Documentação Técnica

Plataforma de Catálogo de Produtos + Painel Administrativo

Stack: React · FastAPI · MongoDB

Versão 1.1 · Fevereiro de 2026

Documento gerado automaticamente

Uso interno · Hidrara Conexões · Matriz Araraquara/SP

Sumário

1	Visão Geral do Projeto
2	O que foi enviado ao GitHub (e o que não foi)
3	Manual de Instalação e Hospedagem
3.1	Hospedagens permitidas e recomendadas
3.2	Banco de dados recomendado
3.3	Armazenamento de imagens (Object Storage)
3.4	Passo a passo: deploy do Backend (Render)
3.5	Passo a passo: deploy do Frontend (Vercel)
3.6	Migração do banco (mongodump / mongorestore)
3.7	Domínio próprio e HTTPS
3.8	Checklist de variáveis de ambiente
4	Documentação Técnica
4.1	Arquitetura da aplicação
4.2	Stack e dependências
4.3	Estrutura de pastas
4.4	Modelos de dados (MongoDB)
4.5	Endpoints da API
4.6	Rotas do frontend
4.7	Fluxo de autenticação (JWT)
4.8	Categorias e produtos (seed)
4.9	Unidades (matriz + filiais)
4.10	Formulário de contato e caixa de entrada
5	Operação diária do painel administrativo
6	Backup e recuperação
7	Segurança
8	Roadmap e evolução

1. Visão Geral do Projeto

Este documento descreve o site institucional e catálogo de produtos da **Hidrara Conexões e Equipamentos Hidráulicos**, empresa fundada em 1991 em Araraquara/SP, distribuidora autorizada Parker Hannifin, Mann-Filter e Donaldson Company. A aplicação foi construída como um sistema **full-stack moderno** que unifica três frentes: site público de apresentação da empresa, catálogo de produtos pesquisável por nome ou código, e painel administrativo protegido para gerenciamento em tempo real do catálogo e das mensagens recebidas pelo formulário de contato.

Objetivos principais

- Consolidar a presença digital da Hidrara com identidade visual profissional e coerente com a marca.
- Permitir que a equipe atualize produtos, fotos, códigos e informações comerciais sem intervenção técnica.
- Facilitar a busca de peças pelos clientes por meio de códigos originais (P954208, WK10002, PERI-340-10C etc.).
- Centralizar leads de contato em uma caixa de entrada administrativa, evitando perda de oportunidades.
- Manter todos os dados institucionais corretos: matriz em Araraquara/SP, 8 unidades no Brasil, telefone único (16) 3508-1300.

2. O que foi enviado ao GitHub (e o que não foi)

Ao usar a opção **Save to GitHub**, apenas o código-fonte da aplicação é versionado. O banco de dados MongoDB, os arquivos enviados via upload e chaves secretas **não** são armazenados no repositório. Isso é a prática correta: dados sensíveis e de produção vivem em serviços dedicados; o repositório contém apenas o que é reproduzível.

Item	Vai para o GitHub?
Backend Python (FastAPI, server.py)	Sim
Frontend React (src/, public/, package.json)	Sim
Imagens estáticas (logo, fachada, clientes, 24 fotos de produtos)	Sim (public/hidrara/)
Arquivo .env (senhas, chaves JWT, credenciais)	NÃO — deve ser recriado no servidor
Documentos institucionais (PRD.md, credenciais de teste)	Sim (pasta memory/)
Banco MongoDB (produtos cadastrados, admin, mensagens)	NÃO
Arquivos enviados via upload no painel admin	NÃO — ficam no Object Storage

Importante: Ao migrar para uma nova hospedagem, você tem duas opções para os dados de produtos. **(1)** Reaproveitar o *seed* automático embutido no código, que recria o usuário admin e 24 produtos-modelo na primeira execução. **(2)** Exportar o banco atual via *mongodump* e importar no novo servidor com *mongorestore* — recomendado se você já cadastrou produtos próprios pelo painel.

3. Manual de Instalação e Hospedagem

3.1 Hospedagens permitidas e recomendadas

A aplicação é composta por três camadas independentes: **frontend React**, **backend FastAPI (Python)** e **banco de dados MongoDB**. Cada camada pode ser hospedada em provedores diferentes, o que aumenta a robustez, permite escalonamento independente e geralmente reduz custos.

Camada	Recomendado (melhor custo/benefício)	Alternativas suportadas
Frontend React	Vercel — plano gratuito, deploy contínuo a partir do GitHub, HTTPS incluso.	Netlify · Cloudflare Pages · GitHub Pages · AWS Amplify.
Backend FastAPI	Render.com — plano gratuito com sleep automático, ou Railway (US\$ 5/mês).	Fly.io · Heroku · DigitalOcean App Platform · AWS EC2/ECS · Google Cloud Run.
Banco MongoDB	MongoDB Atlas M0 — 512 MB gratuitos, oficial, escalável sob demanda.	DigitalOcean Managed MongoDB · AWS DocumentDB · MongoDB self-hosted em VPS.
Object Storage (imagens)	Cloudflare R2 — 10 GB gratuitos, sem custo de saída de dados.	AWS S3 · Backblaze B2 · DigitalOcean Spaces · Google Cloud Storage.

Combinação recomendada: Vercel (frontend) + Render (backend) + MongoDB Atlas (dados) + Cloudflare R2 (imagens). Custo inicial: R\$ 0 (todos os serviços têm plano gratuito com quotas confortáveis). Na fase de crescimento, upgrade escala suavemente para ~US\$ 15 a 30/mês. Todos aceitam deploy diretamente a partir do repositório GitHub.

3.2 Banco de dados recomendado

A aplicação foi construída sobre **MongoDB 6+**. O banco é **obrigatoriamente MongoDB** (NoSQL) porque a estrutura de coleções, os índices únicos por código de produto e a persistência de campos flexíveis (descrição livre, URLs de imagem, categorias, mensagens de contato) são otimizados para esse modelo.

Coleções utilizadas:

- **users** — administradores do painel (senha em bcrypt).
- **products** — 24 produtos padrão + os cadastrados no painel.
- **categories** — 8 categorias iniciais.
- **files** — metadados dos arquivos enviados via upload de imagem.
- **messages** — mensagens do formulário de contato público (novidade v1.1).
- **settings** — versão do seed (regeneração controlada).

Serviço recomendado: MongoDB Atlas. É o serviço oficial da MongoDB, com plano gratuito M0 (512 MB, suficiente para milhares de produtos), e escalonamento sob demanda. Cadastro em <https://www.mongodb.com/atlas>. O cluster é criado em ~3 minutos e a connection string fica pronta para colar na variável MONGO_URL.

3.3 Armazenamento de imagens (Object Storage)

As imagens dos produtos podem ser fornecidas de duas formas pelo painel administrativo: colando uma URL externa (por exemplo, um link do site do fabricante) ou **fazendo upload direto** do arquivo. No ambiente Emergent atual, o upload usa o Object Storage nativo. Para hospedagem própria, o

recomendado é substituir por **Cloudflare R2** (compatível com API S3, 10 GB gratuitos, sem custo de saída de dados) ou **AWS S3**. Se o volume de uploads for pequeno (menos de ~5 GB), é possível gravar os arquivos em disco no próprio backend — mas isso não é recomendado em plataformas serverless como Render Free, onde o disco é efêmero.

3.4 Passo a passo: deploy do Backend (Render.com)

1. Crie uma conta em <https://render.com> e conecte sua conta GitHub.
2. Clique em **New +** → **Web Service** e escolha o repositório do projeto.
3. Configure os parâmetros:
 - **Name:** hidrara-api
 - **Root Directory:** backend
 - **Environment:** Python 3
 - **Build Command:** `pip install -r requirements.txt`
 - **Start Command:** `uvicorn server:app --host 0.0.0.0 --port $PORT`
4. Em **Environment**, adicione as variáveis listadas na seção 3.8.
5. Clique em **Create Web Service**. Em cerca de 3 minutos o backend estará no ar em uma URL como <https://hidrara-api.onrender.com>.
6. Acesse <https://hidrara-api.onrender.com/api/> para confirmar que retorna `{"service":"Hidrara API","status":"ok"}`.

3.5 Passo a passo: deploy do Frontend (Vercel)

1. Crie uma conta em <https://vercel.com> e conecte sua conta GitHub.
2. Clique em **Add New Project** e escolha o repositório.
3. Configure:
 - **Framework Preset:** Create React App
 - **Root Directory:** frontend
 - **Build Command:** `yarn build`
 - **Output Directory:** build
4. Em **Environment Variables**, defina:

```
REACT_APP_BACKEND_URL = https://hidrara-api.onrender.com
```

5. Clique em **Deploy**. Em ~2 minutos o site estará em uma URL como <https://hidrara.vercel.app>.

3.6 Migração do banco (mongodump / mongorestore)

Se você já cadastrou produtos próprios no painel antes da migração, é preciso exportar e restaurar esses dados no novo MongoDB Atlas. O procedimento leva menos de cinco minutos:

Exportar (do MongoDB atual):

```
mongodump --uri="mongodb://usuario:senha@host:porta/nome_do_banco" --out=./backup_hidrara
```

Restaurar (para o MongoDB Atlas novo):

```
mongorestore --uri="mongodb+srv://usuario:senha@cluster.mongodb.net/nome_do_banco"
./backup_hidrara
```

As connection strings completas você obtém no painel do Atlas em **Connect** → **Drivers**. Depois de restaurar, atualize a variável `MONGO_URL` do backend para apontar para o Atlas e reinicie o serviço.

3.7 Domínio próprio e HTTPS

Para associar um domínio (por exemplo, `hydrara.com.br`) ao site:

- Em Vercel/Netlify: **Settings** → **Domains** → **Add Domain** e siga as instruções de DNS (CNAME ou A record).
- HTTPS é fornecido automaticamente (Let's Encrypt), sem custo.
- Aponte um subdomínio como `api.hidrara.com.br` para o backend no Render.
- Atualize `REACT_APP_BACKEND_URL` no Vercel para `https://api.hidrara.com.br` e refaça o deploy do frontend.

3.8 Checklist de variáveis de ambiente

Backend (Render/Railway/Fly):

Variável	Descrição	Exemplo
MONGO_URL	String de conexão do MongoDB.	<code>mongodb+srv://user:pass@cluster.net</code>
DB_NAME	Nome do banco de dados.	<code>hydrara</code>
JWT_SECRET	Chave secreta para tokens (32+ caracteres aleatórios).	gerar com: <code>openssl rand -hex 32</code>
ADMIN_EMAIL	E-mail do admin inicial (criado no primeiro deploy).	<code>admin@hydrara.com.br</code>
ADMIN_PASSWORD	Senha do admin inicial (mín. 12 caracteres).	senha forte, difícil de adivinhar
CORS_ORIGINS	URL do frontend (vírgula-separado se >1).	<code>https://hydrara.vercel.app</code>
APP_NAME	Identificador da app (paths do storage).	<code>hydrara</code>
EMERGENT_LLM_KEY	Chave do Object Storage Emergent — só necessária se hospedar na plataforma Emergent.	opcional em produção externa

Frontend (Vercel/Netlify):

Variável	Descrição	Exemplo
REACT_APP_BACKEND_URL	URL pública do backend.	<code>https://api.hidrara.com.br</code>

4. Documentação Técnica

4.1 Arquitetura da aplicação

A aplicação segue o padrão **Single Page Application** (SPA) com backend REST desacoplado:

```
[Navegador do usuário]
  ■ HTTPS
[Frontend React – hospedado na Vercel]
  ■ HTTPS + JWT (cookie ou Bearer)
[Backend FastAPI – hospedado no Render]
  ■ Motor async driver
[MongoDB Atlas] ■ [Object Storage: R2 ou S3 – imagens de produtos]
```

O frontend consome exclusivamente a API do backend via a variável `REACT_APP_BACKEND_URL`. Todas as rotas do backend começam com o prefixo `/api/`. A autenticação usa **JWT em cookies HttpOnly**, com fallback via header `Authorization: Bearer` para ambientes que bloqueiam cookies cross-site.

4.2 Stack e dependências

Camada	Tecnologia	Versão
Frontend	React (Create React App)	19.x
Frontend	React Router	7.x
Frontend	TailwindCSS	3.4
Frontend	shadcn/ui (Radix + Tailwind)	latest
Frontend	Axios	1.x
Frontend	lucide-react (ícones)	latest
Frontend	sonner (toasts)	latest
Backend	FastAPI	0.11x
Backend	Motor (MongoDB async driver)	3.x
Backend	PyJWT + bcrypt	latest
Backend	Pydantic	2.x
Banco	MongoDB	6 ou superior

4.3 Estrutura de pastas

```
/app
■■■ backend/
■   ■■■ server.py    # API FastAPI completa
■   ■■■ requirements.txt
■   ■■■ .env         # secrets (NÃO versionar)
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ App.js
```

```
■ ■ ■■■ pages/      # PublicSite, AdminLogin, AdminDashboard
■ ■ ■■■ components/ # Navbar, ProductCard etc.
■ ■ ■■■ context/    # AuthContext (JWT)
■ ■ ■■■ lib/        # api.js (axios)
■ ■ ■■■ index.css
■ ■■■ public/hidrara/ # logos + fachada + 24 fotos de produtos
■ ■■■ package.json
■ ■■■ .env
■■■ memory/          # PRD.md, credenciais de teste
```

4.4 Modelos de dados (MongoDB)

products

```
{
  "id": "uuid-v4",
  "code": "P954208",      # único, indexado
  "name": "Filtro Hidráulico Parker Racor",
  "category": "filtros",  # slug
  "description": "...",
  "image_url": "/hidrara/products/P954208.png",
  "brand": "Parker Racor",
  "stock": "available|on_request|out",
  "featured": true,
  "created_at": "ISO-8601",
  "updated_at": "ISO-8601"
}
```

categories

```
{ "id": "uuid", "name": "Hidráulica", "slug": "hidraulica" }
```

users

```
{
  "id": "uuid",
  "email": "admin@hidrara.com.br",
  "password_hash": "bcrypt(...)",
  "name": "Administrador",
  "role": "admin",
  "created_at": "ISO-8601"
}
```

messages

```
{
  "id": "uuid",
  "name": "João da Silva",
  "email": "joao@empresa.com",
  "phone": "(16) 99999-0000",
  "message": "Preciso de cotação para 10 filtros P954208",
  "read": false,      # marcada como lida ao abrir no painel
  "created_at": "ISO-8601"
}
```

4.5 Endpoints da API

Método	Rota	Descrição	Auth?
GET	/api/	Health check	não
GET	/api/site/info	Dados institucionais (unidades, MVV, parceiros)	não
GET	/api/categories	Lista categorias	não
POST	/api/categories	Cria categoria	sim
DELETE	/api/categories/{id}	Remove categoria	sim
GET	/api/products	Lista produtos (?q= busca, ?category= filtra)	não

Método	Rota	Descrição	Auth?
GET	/api/products/{id}	Detalhe do produto	não
POST	/api/products	Cria produto	sim
PUT	/api/products/{id}	Atualiza produto	sim
DELETE	/api/products/{id}	Remove produto	sim
POST	/api/upload	Upload de imagem (multipart)	sim
GET	/api/files/{path:path}	Serve imagem enviada	não
POST	/api/contact	Recebe mensagem do formulário público	não
GET	/api/contact	Lista mensagens recebidas	sim
PATCH	/api/contact/{id}/read	Marca mensagem como lida	sim
DELETE	/api/contact/{id}	Remove mensagem	sim
POST	/api/auth/login	Login (retorna JWT + cookie)	não
POST	/api/auth/logout	Logout (limpa cookies)	sim
GET	/api/auth/me	Dados do usuário logado	sim

4.6 Rotas do frontend

- **/** — Site público completo (hero, distribuidores autorizados, missão/visão/valores, setores, produtos com busca e filtros, projetos, unidades, clientes reais, marcas parceiras, depoimentos, formulário de contato).
- **/admin/login** — Tela de login do administrador.
- **/admin** — Painel com abas Produtos e Mensagens (protegido).

4.7 Fluxo de autenticação (JWT)

1. Usuário submete e-mail e senha no formulário de login.
2. Backend valida a senha com **bcrypt** e emite dois tokens:
 - **access_token** — validade de 8 horas.
 - **refresh_token** — validade de 7 dias.
3. Tokens são enviados como cookies **HttpOnly + Secure + SameSite=None** e também no corpo da resposta.
4. O frontend armazena o **access_token** em `localStorage` como fallback e o envia via header **Authorization: Bearer** em todas as chamadas subsequentes.
5. Rotas protegidas verificam o token antes de executar. Sessão expirada retorna 401 e força novo login.

4.8 Categorias e produtos (seed)

Categorias iniciais (8):

Hidráulica · Pneumática · Mangueiras e Conexões · Filtros e Filtração · Vedações · Rolamentos e Mancais · Chicotes Elétricos · Ferragens.

Produtos iniciais (24):

Bombas WK10002, WK11001X, WP11102. Válvulas X002103, X002277, X220184, X220185. Chicote X770733. Filtros Parker Racor P954208, P954869, P956639, P958225, P958404. Mangueiras/conexões PERI-340-10C, PER-67. Atuador PF420. Rolamentos REL-100, REL-802, REL-803. Vedações REC-153, REC-154, REC-672, REC-888, REC-906. Cada produto possui foto real vinculada.

Toda vez que o backend inicia, o método `seed_products()` verifica a versão do seed na coleção `settings`. Se estiver desatualizada, o catálogo padrão é regenerado. Em produção, você pode manter o seed rodando (garante existência dos produtos padrão) ou removê-lo depois de cadastrar seu próprio catálogo definitivo.

4.9 Unidades (matriz + filiais)

Papel	UF	Cidade	Endereço
MATRIZ	SP	Araraquara	Av. Engenheiro Camilo Dinucci, 4101 · Jardim Arco-Íris · CEP 14.808-100
FILIAL	SP	Sertãozinho	Av. Nelson Benedito Machado Pontal, 724 · CINEP · CEP 14.176-110
FILIAL	MS	Dourados	Av. Marcelino Pires, 6750 · Jardim Márcia · CEP 79.841-000
FILIAL	MS	Três Lagoas	Av. Clodoaldo Garcia, 1909 · Vila Guanabara · CEP 79.621-432

Papel	UF	Cidade	Endereço
FILIAL	MS	Ribas do Rio Pardo	Av. Aureliano Moura Brando, 1945 · Parque Estoril · CEP 79.184-160
FILIAL	BA	Teixeira de Freitas	Consulte via (16) 3508-1300
FILIAL	MA	Açailândia	Consulte via (16) 3508-1300
FILIAL	SC	Itajaí	Consulte via (16) 3508-1300

Telefone único para todas as unidades: **(16) 3508-1300**. WhatsApp unificado no site aponta para <https://wa.me/551635081300> com mensagem padrão de atendimento.

4.10 Formulário de contato e caixa de entrada

Frontend: o formulário na seção "Contato" do site público envia um POST para `/api/contact` com nome, telefone, e-mail e mensagem. Após envio bem-sucedido, exibe toast de confirmação e limpa os campos. Validação de e-mail ocorre no navegador (HTML5) e no backend (Pydantic + EmailStr).

Backend: mensagens são gravadas na coleção `messages` com `read=false` por padrão. Endpoints de leitura (GET `/api/contact`), marcação como lida (PATCH `/read`) e exclusão (DELETE) exigem autenticação.

Painel admin: uma nova aba "Mensagens" lista todas as entradas ordenadas pela mais recente, com contador de não lidas em badge amarelo. Clique em uma mensagem para abrir o detalhe completo, marcar automaticamente como lida, responder por e-mail (mailto) ou excluir.

5. Operação diária do painel administrativo

Acesso ao painel

URL: `https://seu-dominio.com/admin/login`

E-mail padrão: `admin@hidrara.com.br`

Senha padrão: `Hidrara@2026`

Recomendação forte: troque a senha padrão logo no primeiro acesso, alterando a variável de ambiente `ADMIN_PASSWORD` no servidor e reiniciando o backend. Como evolução futura, uma rota para trocar senha diretamente no painel pode ser implementada.

Aba "Produtos"

- Clique em **Novo produto** para cadastrar.
- Campos obrigatórios: código, nome, categoria.
- Escolha o modo de imagem: URL externa ou upload direto.
- Marque **Destaque** para exibir com selo no site público.
- Cada linha tem botões de editar (lápiz) e remover (lixeira).
- Barra de busca aceita nome ou código parcial.
- Filtro por categoria mostra apenas os itens da categoria escolhida.

Aba "Mensagens"

- Cada nova mensagem aparece com marcador amarelo (não lida).
- Clique para abrir o detalhe; a mensagem é automaticamente marcada como lida.
- Botão **Responder por e-mail** abre o cliente de e-mail com o destinatário preenchido.
- Botão **Excluir** remove a mensagem permanentemente.
- Contador de não lidas fica visível na aba, ajudando a priorizar respostas.

6. Backup e recuperação

O MongoDB Atlas oferece backups automáticos a partir do plano M10 — método recomendado para produção real. No plano gratuito M0, você pode automatizar um dump semanal via GitHub Actions ou cron externo:

```
# Executar toda quinta-feira às 03h
0 3 * * 4 mongodump --uri="$MONGO_URL" --archive="backup_$(date +%F).gz" --gzip
```

Armazene os arquivos gerados em um bucket separado (Cloudflare R2, S3 ou Google Drive). Restauração usa `mongorestore --gzip --archive=arquivo.gz`.

7. Segurança

- Senhas armazenadas com **bcrypt**, nunca em texto plano.
- Cookies de sessão marcados como **HttpOnly + Secure + SameSite=None**.
- Endpoints de escrita (POST/PUT/DELETE) exigem token válido — sem autenticação retornam 401.
- `JWT_SECRET` deve ser único por ambiente e ter no mínimo 32 caracteres aleatórios.
- Habilite HTTPS em todos os hosts — Vercel, Render e Netlify fazem isso por padrão.

- Limite CORS_ORIGINS apenas ao(s) domínio(s) do frontend em produção. Evite "*".
- Faça rotação da senha do administrador a cada 90 dias.
- O formulário de contato é público — considere adicionar reCAPTCHA se começar a receber spam.

8. Roadmap e evolução

Sugestões de próximas melhorias, ordenadas por impacto no negócio:

- 1. Sistema de leads/cotações estruturado.** Além do formulário livre, criar botão "Solicitar cotação" em cada produto que gera um lead rastreável no painel — permite medir quais produtos convertem mais e priorizar estoque e mídia paga.
- 2. Importação em massa via CSV.** Upload de planilha com centenas de códigos, nomes e imagens de uma só vez, evitando cadastro manual item a item.
- 3. Página "Sobre nós" dedicada.** Página institucional com história completa (1991 → hoje), certificações, equipe, notícias.
- 4. SEO + Open Graph.** Metadados por categoria e produto para captar tráfego orgânico do Google e boa aparência ao compartilhar em WhatsApp/redes sociais.
- 5. Google Analytics + rastreamento de conversão.** Medir cliques nos botões WhatsApp e e-mail; entender de onde vêm os melhores leads.
- 6. Multi-usuário no admin.** Perfil vendedor (só leitura), gerente (pode editar), administrador (tudo). Log de auditoria de quem alterou o quê.
- 7. Integração com WhatsApp Business API.** Auto-responder no primeiro contato e roteamento por filial.

Fim do documento. Em caso de dúvidas técnicas, consulte o repositório GitHub do projeto, os arquivos memory/PRD.md e memory/test_credentials.md, ou entre em contato com o time de desenvolvimento.